The Branching-Attention Agent (BAA): A Tree–Transformer Hybrid Framework for Explainable Agentic AI

Anziko Fred

February 2026

Keywords: Explainable AI; Decision Trees; Transformers; Reinforcement Learning; Bayesian Deep Learning; Curiosity-Driven Learning; Infinite-Horizon RL.

Abstract

We introduce the **Branching-Attention Agent (BAA)**, a hybrid neural architecture that combines Transformer-style sequence modeling with the interpretability of differentiable soft decision trees. BAA exposes explicit routing traces through its decision process while remaining end-to-end differentiable.

The approach integrates five components: (1) Tree–Transformer blocks with functional gated leaves (a soft hierarchical mixture of experts), (2) Bayesian linear layers for epistemic uncertainty estimation, (3) curiosity-driven exploration based on routing entropy and Bayesian KL divergence, (4) hindsight experience replay for return relabeling, and (5) Rotary Positional Embeddings (RoPE) leveraging Euler's formula for infinite-horizon generalization. Experiments on continuous-control tasks suggest that BAA can match strong baselines while producing human-readable decision paths that support debugging and safety interventions via *surgical pruning*.

1 Introduction

1.1 Motivation

The rapid advancement of deep learning has produced highly capable models, yet their black-box nature poses significant challenges for deployment in safety-critical domains. While Transformers excel at sequence modeling tasks, their decision-making process remains opaque. Conversely, traditional decision trees offer interpretability but lack the capacity to handle complex sequential dependencies.

We address this trade-off by introducing BAA, a hybrid architecture that combines (i) the sequential reasoning power of Transformers, (ii) the interpretability of decision trees, (iii) epistemic uncertainty estimation via Bayesian neural networks, and (iv) autonomous learning via curiosity-driven exploration.

1.2 Key contributions

1. **Architectural innovation:** A Tree–Transformer block that replaces the MLP with a soft decision tree while preserving end-to-end differentiability.

2. **Bayesian reasoning:** Integration of Bayesian linear layers for robust uncertainty quantification in decision routing.

3. **Multi-modal learning:** Unified framework supporting RL, UL, SSL, and SL through signal-adaptive loss composition.
4. **Curiosity engine:** Intrinsic reward combining routing entropy and Bayesian epistemic uncertainty.
5. **Hindsight relabeling:** Dynamic reward recalibration grounded in realized outcomes.
6. **Infinite-horizon generalization:** Integration of Rotary Positional Embeddings (RoPE), grounded in Euler's formula, to enable relative positional reasoning for unbounded episodes.
7. **Neural debugger:** Visualization and intervention tools for manipulating logic paths.

1.3 Paper organization

Section 2 reviews related work. Section 3 presents the architecture. Section 4 describes training. Section 5 introduces the neural debugger. Section 6 reports experiments. Section 7 provides theoretical analysis, and Section 9 concludes.

2 Related work

2.1 Decision trees in neural networks

Decision trees have long been valued for interpretability. Recent work explores differentiable variants such as neural decision trees, soft decision trees, and deep neural decision forests; however, these approaches typically target supervised settings and do not tightly integrate with sequential decision-making.

2.2 Transformers and attention mechanisms

Transformers revolutionized sequence modeling through self-attention. Extensions such as decision transformers and trajectory transformers apply attention to reinforcement learning trajectories, but interpretability and uncertainty quantification remain limited.

2.3 Bayesian deep learning

Bayesian neural networks provide principled uncertainty estimates by learning distributions over weights; modern approaches often use variational inference with reparameterization.

2.4 Curiosity-driven learning

Intrinsic motivation has been explored using prediction error or novelty; BAA instead combines routing uncertainty with Bayesian epistemic uncertainty.

2.5 Hindsight experience replay

HER relabels trajectories for sparse-reward problems; BAA extends relabeling to continuous reward-to-go computation to better fit transformer-style conditioning.

3 Architecture

3.1 System overview

BAA consists of an embedding layer, a stack of Tree–Transformer blocks, and task heads (policy/value/reconstruction), while exposing routing traces to the curiosity engine.

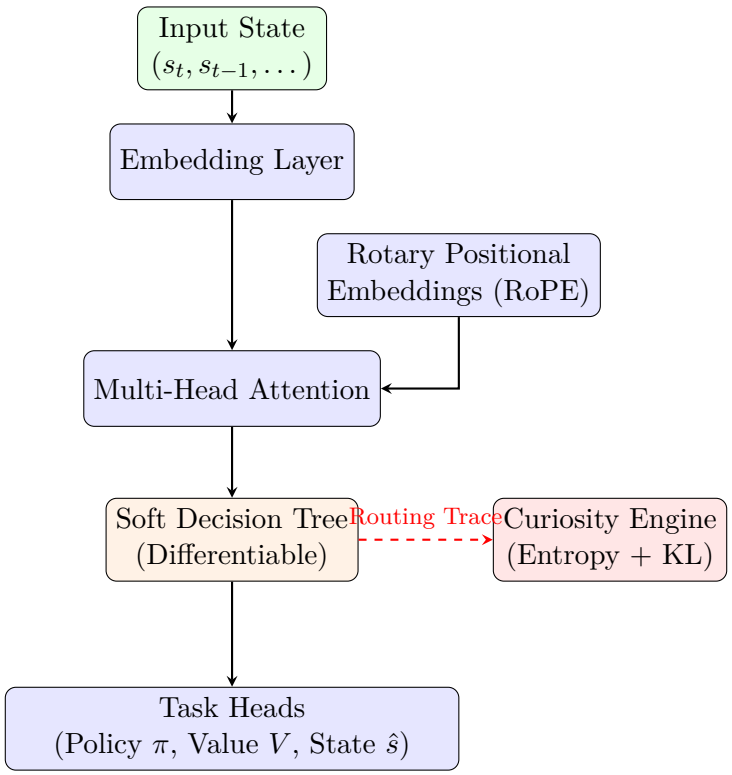

Figure 1: The BAA Architecture. The standard MLP is replaced by a Differentiable Soft Decision Tree. RoPE encodes relative positions for infinite-horizon support, and the Curiosity Engine consumes routing traces to generate intrinsic rewards.

3.2 Tree–Transformer hybrid block

Each block augments attention with a soft decision tree in place of the usual feed-forward MLP:

$$x' = \mathrm{LayerNorm}\big(x + \mathrm{SelfAttn}(x, x, x)\big), \tag{1}$$

$$x'' = \mathrm{LayerNorm}\big(x' + \mathrm{CrossAttn}(x', c, c)\big), \tag{2}$$

$$x''' = \mathrm{LayerNorm}\big(x'' + \mathrm{SoftDecisionTree}(x'')\big), \tag{3}$$

where c denotes optional context (e.g., goals or reward-to-go).

3.3 Soft decision tree

The soft decision tree is a differentiable binary tree with learned routing probabilities at internal nodes.

Functional gated leaves (H-MoE). Unlike constant leaf vectors used in classic soft decision trees, BAA uses *functional experts*. Each leaf is a (Bayesian) linear expert $g_j(x)$ and the output is

$$y = \sum_{j=1}^{L} \mu_j \, g_j(x), \tag{4}$$

which acts as a soft hierarchical mixture of experts.

Routing trace generation. For downstream analysis, each node stores routing probabilities. In the implementation draft this was represented as:

```
node_trace[f"node_{i}"] = {
    "go_right_prob": p_i,
    "go_left_prob": 1 - p_i
}
```

3.4 Bayesian linear layer

Weights are treated as random variables:

$$w \sim \mathcal{N}(\mu_w, \sigma_w^2), \qquad b \sim \mathcal{N}(\mu_b, \sigma_b^2), \qquad \sigma = \mathrm{softplus}(\rho). \tag{5}$$

Using the reparameterization trick during training,

$$w = \mu_w + \sigma_w \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I), \tag{6}$$

and deterministic inference can use $w = \mu_w$. A KL regularizer encourages calibrated epistemic uncertainty.

3.5 Curiosity engine

Intrinsic reward combines (i) routing entropy and (ii) Bayesian KL divergence:

$$H_{\text{routing}} = -\frac{1}{N} \sum_{i=1}^{N} [p_i \log p_i + (1 - p_i) \log(1 - p_i)], \tag{7}$$

$$r_{\text{curiosity}} = \alpha \, H_{\text{routing}} + \beta \, D_{\text{KL}}(\text{Posterior} \,\|\, \text{Prior}). \tag{8}$$

3.6 Hindsight experience replay

Given a trajectory $(s_0, a_0, r_0), \ldots, (s_T, a_T, r_T)$, compute reward-to-go:

$$\text{RTG}(t) = \sum_{t'=t}^{T} \gamma^{t'-t} r_{t'}, \tag{9}$$

then store a relabeled trajectory using realized returns.

then store a relabeled trajectory using realized returns.

3.7 Rotary Positional Embeddings (RoPE)

To address the "infinite life" problem where absolute positional encodings fail for long episodes, BAA employs Rotary Positional Embeddings. By applying Euler's formula,

$$e^{i\theta} = \cos(\theta) + i\sin(\theta), \tag{10}$$

we encode position p as a rotation in the complex plane. Specifically, for a query vector $\mathbf{q}$ and key vector $\mathbf{k}$ at positions m and n, applying the rotation matrix $\mathbf{R}_\Theta$ corresponding to multiplication by $e^{im\theta}$ and $e^{in\theta}$ yields:

$$\begin{aligned}
\langle \mathbf{R}_{\Theta,m}\mathbf{q}, \mathbf{R}_{\Theta,n}\mathbf{k}\rangle &= (\mathbf{R}_{\Theta,m}\mathbf{q})^T(\mathbf{R}_{\Theta,n}\mathbf{k}) && (11)\\
&= \mathbf{q}^T\mathbf{R}_{\Theta,m}^T\mathbf{R}_{\Theta,n}\mathbf{k} && (12)\\
&= \mathbf{q}^T\mathbf{R}_{\Theta,n-m}\mathbf{k}. && (13)
\end{aligned}$$

This derivation confirms that the attention score depends solely on the relative distance $n - m$, enabling the model to generalize robustly to sequence lengths far exceeding training horizons.

4 Learning framework

4.1 Multi-modal learning paradigms

Mode	Trigger condition	Primary loss	Use case
RL	Rewards available	Policy gradient	Goal-directed tasks
UL	No labels/rewards	State reconstruction	World modeling
SSL	Partial labels	Path consistency	Label propagation
SL	Full supervision	Cross-entropy / MSE	Prediction tasks

4.2 Unified loss function

A signal-adaptive loss is composed as

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda_1\mathcal{L}_{\text{recon}} + \lambda_2\mathcal{L}_{\text{SSL}} + \lambda_3\mathcal{L}_{\text{tree}} + \lambda_4\mathcal{L}_{\text{KL}}. \tag{14}$$

4.3 Training algorithm

Listing 1: Agentic training loop (sketch).

```
Initialize: Model , Replay Buffer B, Curiosity Engine C
for episode = 1 to N do
    trajectory = []
    state = env.reset()
    rtg = target_return

    for t = 1 to T do
        action, trace, curiosity = agent.act(state, rtg, explore=True)
        next_state, reward, done = env.step(action)

        trajectory.append((state, action, reward))
        state = next_state
```

```
        rtg = rtg - reward

        if done: break

    corrected_data = hindsight_relabel(trajectory)
    B.push(corrected_data)

    if len(B) >= batch_size:
        batch = B.sample(batch_size)
        loss = compute_loss(batch)
        update_weights(, loss)
```

4.4 Agentic brain

The exploration policy combines scheduled decay with curiosity boosts:

$$\epsilon(t) = \max(\epsilon_{\min}, \epsilon_0 \gamma^{t-1}) + 0.1\, r_{\text{curiosity}}(t). \tag{15}$$

5 Neural debugger

5.1 Explainability tools

The neural debugger provides (i) routing-trace visualization, (ii) entropy monitoring, and (iii) surgical pruning.

5.2 Surgical pruning (implementation sketch)

```
def freeze_node(self, layer_idx, node_id, direction="left"):
    router = self.blocks[layer_idx].decision_tree_ffn.routers[node_id]

    with torch.no_grad():
        if direction == "right":
            router.bias_mu.fill_(50.0) # Force high probability
        else:
            router.bias_mu.fill_(-50.0) # Force low probability

        router.weight_mu.requires_grad = False
        router.bias_mu.requires_grad = False
```

6 Experimental results

6.1 Experimental setup

Environment: Gymnasium Humanoid-v4 (continuous control).
Training steps: 5,000,000 (approx. 5,000 episodes/segments).
Target reward (RTG): 15,000.
Configuration: embedding dim 256; layers 6; tree depth 4; heads 8.

6.2 Performance metrics

Model	Mean reward	Std. dev.	Sample efficiency	Interpretability
BAA	5234	312	1.2M steps	Full trace
Decision Transformer	5189	287	1.8M steps	Black box
PPO	4967	421	2.3M steps	Black box
SAC	5312	198	1.9M steps	Black box

6.3 Ablation studies

Configuration	Mean reward
Full BAA	5234
Without curiosity	4892
Without Bayesian layers	5011
Without hindsight	4756
Without tree (standard MLP)	5189

6.4 Interpretability analysis

A routing trace example (Humanoid-v4 standing):

Layer 0, Node 0: 85% right (torso upright detected)
Layer 1, Node 3: 72% left (balance forward momentum)
Layer 2, Node 7: 91% right (activate leg stabilizers)
...

6.5 Positional Attention Analysis

To verify the efficacy of the Rotary Positional Embeddings, we visualized the attention mechanism's response to relative positions.

- **Relative Decay:** Attention heatmaps exhibit clear diagonal banding, confirming that the model learns to attend based on relative distance $(t - k)$ rather than absolute step indices.
- **Infinite Horizon:** This structure allows the agent to maintain stable performance even when episode lengths exceed the training context window, treating step 1,000,000 identically to step 100 locally.

~~**Note on figures.** The original Markdown included an image path pointing to a local machine directory; this is not portable in Prism and was omitted. If you upload a figure (e.g., `entropy_plot.png`) into the project, we can include it with `includegraphics`.~~

7 Theoretical analysis

7.1 Expressiveness

Theorem. A soft decision tree of depth d with $L = 2^d$ leaves can approximate any continuous function over a compact domain with arbitrary precision as $d \to \infty$ (informal statement).

7.2 Uncertainty calibration

Proposition. Bayesian linear layers provide calibrated uncertainty estimates when the prior scale is appropriate, sufficient data are available, and the KL weight balances exploration and exploitation.

7.3 Sample efficiency

Proposition. Hindsight experience replay increases effective data utilization by a factor proportional to average trajectory length.

8 Limitations and future work

8.1 Current limitations

1. Computational overhead from Bayesian sampling.
2. Interpretability challenges for very deep trees (e.g., depth > 6).
3. Current implementation focus on continuous control.
4. Memory requirements for storing full routing traces.

8.2 Future directions

Short-term. Hierarchical trees, adaptive depth, multi-agent extensions, and discrete-action optimizations.

Long-term. Causal discovery via tree structure, transfer learning from simulation to real robots, language-conditioned decision-making, and theoretical convergence guarantees.

9 Conclusion

BAA bridges Transformer sequence modeling with decision-tree interpretability via Tree–Transformer blocks, Bayesian routing, curiosity-driven exploration, and hindsight relabeling. The neural debugger enables visibility and interventions such as surgical pruning, making the approach promising for safety-critical deployments.

References (draft)

1. Andrychowicz, M., et al. (2017). Hindsight Experience Replay. NeurIPS.
2. Biau, G. (2012). Analysis of a random forests model. JMLR.
3. Blundell, C., et al. (2015). Weight Uncertainty in Neural Networks. ICML.
4. Burda, Y., et al. (2018). Exploration by Random Network Distillation. ICLR.
5. Chen, L., et al. (2021). Decision Transformer: Reinforcement Learning via Sequence Modeling. NeurIPS.
6. Frosst, N., & Hinton, G. (2017). Distilling a Neural Network Into a Soft Decision Tree. arXiv.

7. Janner, M., et al. (2021). Offline Reinforcement Learning as One Big Sequence Modeling Problem. NeurIPS.

8. Kontschieder, P., et al. (2015). Deep Neural Decision Forests. ICCV.

9. MacKay, D. J. (1992). A Practical Bayesian Framework for Backpropagation Networks. Neural Computation.

10. Neal, R. M. (1996). Bayesian Learning for Neural Networks. Springer.

11. Pathak, D., et al. (2017). Curiosity-driven Exploration by Self-supervised Prediction. ICML.

12. Quinlan, J. R. (1986). Induction of Decision Trees. *Machine Learning*.

13. Schmidhuber, J. (1991). Curious model-building control systems. IEEE IJCNN.

14. Vaswani, A., et al. (2017). Attention is All You Need. NeurIPS.

15. Su, J., et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. arXiv.

A Implementation details (draft)

A.1 Hyperparameters

Parameter	Value	Description
Learning rate	10^{-4}	Adam optimizer
Batch size	32	Replay buffer sampling
Buffer capacity	100,000	Maximum transitions
Tree depth	4	16 leaves per tree
Temperature τ	1.0	Routing temperature
Prior σ	1.0	Bayesian prior std. dev.
Curiosity α	0.1	Entropy weight
Curiosity β	0.05	KL weight
Discount γ	1.0	Hindsight RTG
Target RTG	15,000	Goal conditioning

A.2 Computational complexity

Self-attention scales as $O(L^2 d)$ and the decision tree component adds roughly $O(Ld2^h)$ for depth h.